

Ejercicio en Clase

Spark SQL + DataFrames

INSTRUCCIONES:

Dentro del material del curso proporcionado por su catedrático, cargue los archivos `/trump_tweets/donald_data.json` y `/nba/games_details.csv` dentro de una carpeta en su ambiente de trabajo dentro de Databricks.

PREPARACIÓN DEL AMBIENTE

Dentro del material proporcionado, copie el archivo que descargó dentro de una carpeta en su ambiente de trabajo (workspace) de su ambiente personal de Databricks. Esto lo puede lograr de la siguiente manera: seleccione la opción de New → Add or upload data desde el menú del lado izquierdo. En el apartado de **Files** escoja la opción Upload files to a volume. Importe el archivo desde su equipo, y en la parte inferior de los catálogos disponibles, escoja un lugar en donde desea almacenar estos datos. Por ejemplo, usando su workspace siga la ruta de All catalogs → workspace → default → <coloque un nombre de volumen a su elección>. Esto le dará como resultado una ruta de tipo `/Volumes/workspace/default/<my volume>` que podrá referenciar luego en su Notebook.

EJERCICIOS

CARGA DE DATOS Y ANÁLISIS EXPLORATORIO (20 PTS)

SparkSQL nos provee un API muy completo para carga de un dataset. A diferencia de cuando se cargan RDDs con `sc.textFile()`, no necesitamos preocuparnos por el parse o interpretación del formato en que venga nuestra fuente. Conociendo lo básico del formato de la data que queremos cargar podemos cargar virtualmente cualquier dataset. Por el nombre del archivo, conocemos que `donald_data.json` contiene datos en formato JSON, por lo que vamos a usar el método `.json()` del `DataFrameReader` para cargarlo.

```
>>> trump = spark.read.json(working_dir + '/trump_tweets/donald_data.json')
>>> trump.printSchema()
```

Notemos que tiene una estructura muy compleja, así es como el API de twitter entrega los tweets cuando se le hacen consultas. Probablemente hay demasiados campos que para este ejercicio no nos interesan, por lo que vamos a utilizar el método `.select()` para definir un nuevo DataFrame con un subconjunto de las columnas que nos interesan:

```
>>> trump_clean = trump.select(
... 'user.screen_name',
... 'created_at',
... 'full_text',
... 'retweet_count',
... 'favorite_count'
... )
>>> trump_clean.show()
```

Es posible que al correr el comando `.show()`, la consola nos retorne un error similar al siguiente: `UnicodeEncodeError: 'charmap' codec can't encode character u'\u201c'`. Esto es un caso común y es un error que ocurre del lado de la consola de Python, no dentro de Spark. Suele ocurrir cuando queremos imprimir una columna de texto que puede contener caracteres especiales. Podemos hacer un work around definiendo nuestra columna con un encoding ASCII:

```
>>> from pyspark.sql import functions as f
>>> trump_clean = trump.select(
... 'user.screen_name',
... 'created_at',
... f.encode('full_text', 'ascii').cast('string').alias('text'),
... 'retweet_count',
... 'favorite_count'
... )
>>> trump_clean.show()
```

Ahora sí podemos visualizar nuestro `DataFrame`. Note cómo el método `.show()` de manera predeterminada sólo muestra las primeras filas para evitar que se llene la consola de registros si olvidáramos colocar un límite. El método `.show()` tiene tres argumentos: `n`, `truncate` y `vertical`. A veces, como en este caso, la data se muestra mejor con una visualización distinta a la tabular:

```
>>> trump_clean.show(n=3, truncate=False, vertical=True)
```

Ahora utilice el API de Spark para desplegar en pantalla los 3 tweets más retwitteados del dataset. Te puedes basar en la documentación para conocer los diferentes métodos que puedes aplicar a los `DataFrames` y las funciones que puedes usar para definir operaciones.

<https://spark.apache.org/docs/3.3.0/api/python/reference/pyspark.sql/dataframe.html>

Ahora, ¿cómo podría hacerlo usando un query a través de SQL? Esto no es completamente necesario que lo ejecute por su cuenta (aunque está en completa libertad de investigar un poco), consulte a su instructor para hacer esta parte en conjunto.

AGREGACIONES EN SPARK SQL:

Ahora vamos a cargar otro dataset: `games_details.csv`. Dado que sabemos que la data que queremos cargar es un archivo separado por comas, utilizaremos el método `.csv()` del `DataFrameReader`.

```
>>> nba = spark.read.csv('/nba/games_details.csv')
>>> nba.printSchema()
```

A diferencia de cuando cargamos un JSON, los nombres de las columnas los puso automáticamente como `_c0`, `_c1`, ..., `_c27`, y todos los cargó como tipo String. Esto se debe a que en la estructura de un JSON se almacena más información que en un CSV. Quizá debamos ser más explícitos con un par de instrucciones adicionales en el método `read.csv()`:

```
>>> nba = spark.read.csv(
... header = True,
... inferSchema = True,
... path = working_dir + '/nba_games.csv'
... )
```

Antes de avanzar, ¿notó algo diferente al ejecutar el comando anterior? Si únicamente se está definiendo un nuevo `DataFrame`, ¿por qué se ejecutó un Job de Spark?

Si nuevamente ejecutamos **`nba.printSchema()`** ahora sí veremos más información. Además de los nombres de las columnas propiamente asignados podemos ver que el tipo de dato de cada una también fue asignado. Veamos ahora un poco sobre su contenido; utilicemos el método **`nba.show()`** para visualizar los primeros 20 registros. Se recomienda que coloque su consola en pantalla completa para esta acción debido a que el `DataFrame` tiene muchas columnas.

El dataset representa el desempeño de jugadores de la NBA en todos los juegos desde el 2004. Hay varios nombres de campos cuyas siglas nos pueden confundir. Para tener de referencia, tomado de la fuente, a continuación lo que cada columna representa:

- `GAME_ID`: ID del juego
- `TEAM_ID`: ID del equipo

- TEAM_ABBREVIATION: Abreviación de tres letras del equipo
- TEAM_CITY: Ciudad donde el juego se jugó
- PLAYER_ID: ID del jugador
- PLAYER_NAME: Nombre del jugador
- START_POSITION: Posición del jugador (si es nulo, inició en la banca)
- COMMENT: Comentario
- MIN: Minutos jugados
- FGM: (Field Goals Made) Cantidad de canastas en tiempo de juego (excluye las que son por tiros libres)
- FGA: (Field Goals Attempted) Cantidad de intentos de anotación en tiempo de juego
- FG_PCT: (Field Goal Percentage) Porcentaje canastas anotadas por intentos en tiempo de juego
- FG3M: (Three Pointers Made) Cantidad de canastas de 3 puntos
- FG3A: (Three Pointers Attempted) Cantidad de intentos de disparos de 3 puntos
- FG3_PCT: (Three Point Percentage) Porcentaje de anotación de disparos de 3 puntos
- FTM: (Free Throws Made) Canastas anotadas en tiros libres
- FTA: (Free Throws Attempted) Tiros libres realizados
- FT_PCT: (Free Throw Percentage) Porcentaje de acierto en tiros libres
- OREB: (Offensive Rebounds) Cantidad de recuperaciones ofensivas
- DREB: (Defensive Rebounds) Cantidad de recuperaciones defensivas
- REB: (Rebounds) Cantidad de recuperaciones totales
- AST: (Assists) Cantidad de asistencias
- STL: (Steals) Cantidad de robos
- BLK: (Blocked shots) Cantidad de disparos bloqueados
- TO: (Turnovers) Cantidad de recuperaciones de posesión
- PF: (Personal Foul) Cantidad de faltas
- PTS: (Points) Cantidad de puntos anotados por el jugador
- PLUS_MINUS: Diferencia en el marcador el tiempo que el jugador estuvo en la duela

Utilizando las librerías del SparkSQL API, responda las siguientes preguntas:

1. ¿Cuál es la máxima cantidad de puntos anotados por un jugador en un solo juego?
2. En promedio, ¿cuántos puntos anota un jugador por juego?
3. ¿Qué precisión tienen en promedio por juego todos los jugadores de la NBA en lanzamientos en tiempo de juego? (FG_PCT)
4. Si filtramos sólo los juegos de Kobe Bryant, ¿cuántos puntos hace por juego en promedio?, y ¿qué precisión tiene él en promedio por juego en lanzamientos? (FG_PCT)
5. Volvamos a comparar a Kobe con el promedio de jugadores, pero ahora con la cantidad promedio de intentos (FGA) de canasta.

Podemos ver que cuando hablamos sólo de la precisión, Kobe no destaca mucho más del resto de jugadores (+3%), sin embargo hace muchos más puntos por juego que el resto (+270%). Con el resultado del punto número 5, podríamos decir que Kobe anota mucho más porque lo intenta mucho más que el resto. Incluso cuando lo comparamos sólo contra los jugadores que juegan su misma posición:

6. Volvamos a calcular las tres estadísticas, pero ahora filtrando para todos los jugadores que juegen la misma posición que Kobe.

Ahora calculemos una estadística que no está en el dataset. Los cálculos anteriores son agregados por juego, pero la estadística puede mentir un poco considerando que no todos los jugadores tienen los mismos minutos por juego. Calculemos la cantidad de intentos que hacen los jugadores por minuto. Este cálculo tiene un reto mayor, ya que el campo "MIN" del dataset contiene la cantidad de minutos jugados pero es un string con formato "MM:SS". Primero debemos convertir este campo a un número que podamos operar.

7. Calculemos la columna "minutes" del DataFrame que contenga la cantidad de minutos jugados sin considerar los segundos. Esto lo podemos lograr tomando sólo los primeros dos dígitos del string que representan el minuto con la función `substring()`.
8. Ahora podemos calcular la columna "FGA_per_minute".
9. Comparemos el promedio de esta estadística para los tres subconjuntos del dataset con los que estamos trabajando: total de jugadores, guardas (START_POSITION = "G"), y Kobe Bryant.

Esta comparación es brutal y nos da un insight muy interesante sobre Kobe: hace 0.56 lanzamientos por minuto, o de una manera más fácil de interpretar: en promedio hace un lanzamiento por cada 1:46 de juego cuando en promedio otros jugadores de su posición lo hacen cada 2:38.

10. ¿Será Kobe el jugador que tenga más alta esta estadística en un solo juego?

Probablemente, para responder la última pregunta te hayas topado con un caso muy extraño e improbable. ¿Cómo podríamos responder esto? ¿Será que la posición del jugador tiene algo que ver? ¿Kobe jugaba siempre en la misma posición?